JIRA Dashboard – Graph development using SQL Feasibility Study Report

Title: JIRA Dashboard – Graph development using SQL Feasibility Study Report
Date: 06-May-2025
Prepared by: Sidhanth Kamath
Reviewed by: Prakhyath Rai
Version: v1.0

Contents

1. Introduction 3

2. Data Overview 4

3. Feasibility Analysis & Technical Assessment 6

5. Conclusion & Recommendations 15

6.Results: 17

7. Appendix 23

1. Introduction

Objective:

This report presents an integrated data pipeline and analytics solution to evaluate Jira Agile sprint performance. It combines data extraction from Jira's REST API with structured querying using MS SQL and Python. The objective is to provide actionable insights into sprint planning and execution through metrics such as:

- **Story Points Committed vs Completed**
- **Sprint Scope Change**
- **Sprint Rolling Status**
- **Issue Type Breakdown - time spent**
- **Defects - Created vs. Resolved**
- **Average Time in Status**

This framework supports continuous improvement of Agile processes, retrospective analysis, and strategic sprint planning.

Scope:

The analysis encompasses:

- Extraction of board, sprint, and issue data from Jira API (Due to unavailable Board/Project data from the 'Jira_Test' DB)
- Transformation into structured datasets
- Execution of analytical SQL queries in both MS SQL and Python (via pandasql)
- Visualization and export of reports

The solution supports dynamic querying for multiple Jira projects and is extensible to different types of Agile boards and custom Jira configurations.

2. Data Overview

Source of Data:

The data for this analysis is obtained from two primary sources:

A. Jira REST API (via Python Integration)

The Jira API is accessed using the Atlassian-python-API library, which enables programmatic interaction with Jira's Agile functionalities.

- **API Client**: Initialized using jira_url, jira_token, and jira_username
- **Methods Used**:
    - get_all_agile_boards() – Retrieves board metadata (board ID, type, etc.)
    - get_all_sprints_from_board() – Fetches all sprints for a given board
    - get_all_issues_for_sprint_in_board() – Retrieves issue-level data per sprint

These APIs are paginated and handled dynamically to support large datasets.

Storage Location (Local Files):

- jira_board_data.csv: Board-level metadata
- jira_sprint_data.csv: Sprint-level metadata
- jira_master_dashboard_data.csv: Merged dataset for board, sprint, and issues

B. Jira Backend SQL Database (via MS SQL Server)

For organizations with backend access to Jira's relational database, data is directly queried using SQL from Jira 9.0's database schema.

- **Primary Tables**:
    - jiraissue: Main table for all Jira issues
    - project: Maps project IDs to project keys
    - customfield & customfieldvalue: Used to extract story points (customfield)
    - AO_60DB71_SPRINT: Stores sprint metadata
    - AO_60DB71_RAPIDVIEW: Maps boards to projects
    - changegroup & changeitem: Track issue change history (e.g., sprint changes)

These SQL scripts provide powerful direct access for deeper and more historical analytics where API limitations exist.

Data Description:

The data was transformed into structured datasets used in the analysis and reporting process. Below is a description of the final datasets and their key fields:

1. Board Dataset (df_board)

Contains metadata about all boards within the specified project.

- board_id: Unique ID of the board
- board_name: Name of the board (e.g., "AI Scrum Team")
- board_type: Type of Agile board (Scrum, Kanban)
- project_key: Project key from Jira (e.g., "AI")

2. Sprint Dataset (df_sprint)

Provides sprint-level information, including planning dates and goals.

- sprint_id: Unique sprint ID
- sprint_name: Display name of the sprint
- sprint_state: Current state (active, closed)
- sprint_start_date, sprint_end_date, sprint_complete_date: Timeline attributes
- board_id: Foreign key mapping to the board
- sprint_goal: Textual goal set for the sprint

3. Issue Dataset (df_issue)

Contains all issues pulled from each sprint in each board.

- issue_id: Unique identifier of the issue
- issue_key: Readable identifier (e.g., AI-123)
- issue_type: Type (Story, Task, Bug, Sub-task)
- status: Current issue status (Done, In Progress, etc.)
- story_points: Numeric value assigned via custom field
- summary: Brief description/title of the issue
- sprint_id, board_id, project_key: Mapping fields

4. Master Table (df_master)

Final joined dataset used for reporting. It merges board, sprint, and issue datasets to enable comprehensive cross-dimensional analysis.

- **Dimensions**: project_key, board_id, sprint_id, issue_key
- **Metrics**:
    - story_points
    - Issue counts per sprint
    - Completion rates
    - Rolling status categorization
- **Enriched Fields**: Includes derived fields such as sprint category (New vs Rolling)

3. Feasibility Analysis & Technical Assessment

Since Board & Project Co-Related information wasn't available in the 'Jira_Test' that was provisioned to us, we had to pull the data from JIRA API generating Personal Access Token (PAT) . The Python script fetches and processes Jira Agile data using the **Jira Python API** (via the Atlassian-python-API library). It retrieves boards, sprints, and issue details, then performs SQL-like analysis using pandasql.

Class & Class Method:

Method: *initialize_jira()*

- Purpose: Initializes the Jira client using API token.
- Inputs: URL, username, token.
- Returns: Jira API client object.
- Used In: All Jira data fetch operations.

Method: *get_all_boards_paginated(...)*

- Purpose: Retrieves all Scrum boards for a given Jira project.
- Params: project_key, optional board_name and board_type.
- Output: List of boards.
- Use Case: Board-level analysis and sprint mapping.

Method: *get_all_sprint_from_board_paginated(board_id)*

- **Purpose**: Fetches **all sprints** associated with a board (paginated).
- **Output**: List of sprints with metadata (start, end, goal, etc.).
- **Use Case**: Needed for timeline and planning analysis.

Method: *get_all_issues_for_sprint_in_board_paginated(board_id, sprint_id)*

- **Purpose**: Retrieves all **issues per sprint**, per board.
- **Output**: Complete issue list with types, story points, statuses.
- **Use Case**: Sprint-level performance metrics.

Method: *sql_query_master_table(master_df, query)*

- **Purpose**: Executes **SQL queries** using pandasql on in-memory DataFrame.
- **Inputs**:
    - master_df: Merged master issue table.
    - query: SQL query string.
- **Output**: Query result as a new DataFrame.

SQL Query 1 – Story Points Committed vs Completed

Purpose:

Measures **sprint performance** in terms of:

- Issues committed to sprint.
- Issues completed (status = 'Done').
- Story points completed vs. committed.
- Completion %.

Query Logic

```
SELECT
  sprint_name AS Sprint,
  sprint_id,
  count(*) AS Issues_Committed,
  SUM(CASE WHEN status = 'Done' THEN 1 ELSE 0 END) AS Issues_Completed,
  SUM(CASE WHEN status = 'Done' THEN story_points ELSE 0 END) AS
Story_Points_Completed,
  SUM(story_points) AS Story_Points_Committed,
```

```
  SUM(CASE WHEN status = 'Done' THEN story_points ELSE 0 END) /
SUM(story_points) * 100 AS Story_Points_Completed_Percentage
FROM master_table
WHERE issue_type NOT IN ('Sub-task')
GROUP BY sprint_name, sprint_id
```

Output Columns

- Sprint: Sprint name.
- Issues_Committed: Total issues at start.
- Issues_Completed: Issues marked Done.
- Story_Points_Completed: Points delivered.
- Story_Points_Committed: Total planned.
- Story_Points_Completed_Percentage: Efficiency.

SQL Query 2 – Sprint Rolling Status

Purpose

Classifies issues into:

- **New**: Introduced in this sprint.
- **Rolling**: Moved from previous sprint (same board).

Query Logic

```
SELECT sprint_name, category AS sprint_rolling_state, COUNT(issue_key),
SUM(story_points) AS story_points
FROM (
  SELECT *,
      CASE WHEN psn IS NOT NULL THEN 'Rolling' ELSE 'New' END AS Category
  FROM (
    SELECT *,
        LAG(sprint_name) OVER (PARTITION BY issue_key, board_id) AS psn
    FROM master_table
    WHERE issue_type NOT IN ('Sub-task')
  )
)
```

```
GROUP BY 1, 2
```

Output Columns

- sprint_name: Sprint name.
- sprint_rolling_state: New or Rolling.
- COUNT(issue_key): Number of issues in category.
- story_points: Total story points in each category.

SQL Query 3 – Issue Type Breakdown – Time Spent

Purpose

This query provides a breakdown of the total time spent (in hours) on issues, grouped by **project** and **issue type** (e.g., Bug, Story, Task).

Query Logic

```
SELECT
  P.pname AS project_name,
  IT.pname AS issue_type,
  COUNT(DISTINCT JI.ID) AS issue_count,
  SUM(WL.timeworked) / 3600.0 AS total_hours_spent
FROM jiraissue JI
JOIN project P ON P.ID = JI.PROJECT
JOIN issuetype IT ON IT.ID = JI.issuetype
LEFT JOIN worklog WL ON WL.issueid = JI.ID
--WHERE P.pname = 'NGA'  -- replace with your project name
GROUP BY P.pname, IT.pname
ORDER BY total_hours_spent DESC;
```

Tables Used

- jiraissue: Main issue table.
- project: Contains project metadata.
- issuetype: Contains issue type labels.
- worklog: Tracks time logged against issues.

Key Columns

- P.pname: Project name.
- IT.pname: Issue type name.

- WL.timeworked: Time logged in seconds.

Output Columns

- project_name: Name of the project.
- issue_type: Name of the issue type.
- issue_count: Number of distinct issues of that type.
- total_hours_spent: Sum of time logged in hours.

Customizations

- Use the WHERE clause to filter by a specific project (--WHERE P.pname = 'Project Name').

SQL Query 4 – Average Time in Status

Purpose

This query calculates the **average number of days** an issue spends in each status before transitioning to another.

Query Logic

```
WITH status_durations AS (
  SELECT
    ji.id AS issue_id,
    p.pname AS project_name,
    CAST(ci.oldvalue AS VARCHAR) AS status_id,
    cg.created AS status_changed_at,
    LEAD(cg.created) OVER (PARTITION BY ji.id ORDER BY cg.created) AS
next_status_changed_at
  FROM jiraissue ji
  JOIN project p ON p.id = ji.project
  JOIN changegroup cg ON cg.issueid = ji.id
  JOIN changeitem ci ON ci.groupid = cg.id
  WHERE CAST(ci.field AS NVARCHAR(MAX)) = 'status'
   -- AND p.pname = 'NGA'  -- Change to your project
)

SELECT
  sd.project_name AS project_name,
  iss.pname AS status_name,
  COUNT(*) AS transition_count,
```

```
 AVG(DATEDIFF(HOUR, sd.status_changed_at, sd.next_status_changed_at)) /
24.0 AS avg_days_in_status
FROM status_durations sd
JOIN issuestatus iss ON iss.id = sd.status_id
WHERE sd.next_status_changed_at IS NOT NULL
GROUP BY  sd.project_name, iss.pname
ORDER BY avg_days_in_status DESC;
```

Tables Used

- jiraissue: Issues.
- project: Project metadata.
- changegroup: Tracks changes to issues.
- changeitem: Contains field-level change data.
- issuestatus: Maps status IDs to names.

Key Logic

- Uses a CTE status_durations to capture the duration each issue spent in a status.
- Uses the LEAD() function to get the timestamp of the next status change.

Output Columns

- project_name: Name of the project.
- status_name: Name of the issue status (e.g., In Progress, Done).
- transition_count: Number of times this status was entered.
- avg_days_in_status: Average time in status, in days.

SQL Query 5 – Defects - Created vs Resolved

Purpose

This query compares the **number of defects (Bugs)** created and resolved weekly.

Tables Used

- jiraissue: Issue data (filtered to Bugs).
- issuetype: Issue type mapping.

Query Logic

```
WITH created_issues AS (
  SELECT
    DATEPART(YEAR, ji.created) AS year,
    DATEPART(WEEK, ji.created) AS week,
    COUNT(*) AS created_count
  FROM jiraissue ji
  JOIN issuetype it ON it.id = ji.issuetype
  WHERE it.pname = 'Bug'
  GROUP BY DATEPART(YEAR, ji.created), DATEPART(WEEK, ji.created)
),
resolved_issues AS (
  SELECT
    DATEPART(YEAR, ji.resolutiondate) AS year,
    DATEPART(WEEK, ji.resolutiondate) AS week,
    COUNT(*) AS resolved_count
  FROM jiraissue ji
  JOIN issuetype it ON it.id = ji.issuetype
  WHERE it.pname = 'Bug' AND ji.resolutiondate IS NOT NULL
  GROUP BY DATEPART(YEAR, ji.resolutiondate), DATEPART(WEEK,
ji.resolutiondate)
)
SELECT
  COALESCE(c.year, r.year) AS year,
  COALESCE(c.week, r.week) AS week,
  ISNULL(c.created_count, 0) AS created_issues,
  ISNULL(r.resolved_count, 0) AS resolved_issues
FROM created_issues c
FULL OUTER JOIN resolved_issues r
  ON c.year = r.year AND c.week = r.week
ORDER BY year, week asc;
```

Key Logic

- Two CTEs (created_issues, resolved_issues) group Bugs by year and week.
- A FULL OUTER JOIN ensures weeks with only created or resolved issues are included.

Output Columns

- year: Year of creation or resolution.
- week: Week number.
- created_issues: Number of bugs created.
- resolved_issues: Number of bugs resolved.

Customizations

- Adjust it.pname = 'Bug' for different issue types if needed.
- Can add project filters if needed.

SQL Query 6 – Sprint Scope Change

Purpose

This query identifies issues that were **added to**, **removed from**, or **unchanged** in a sprint — helping track scope changes.

Query Logic

```
WITH issue_sprint_mapping AS (
  SELECT
    cg.issueid,
    CAST(SUBSTRING(CAST(ci.newvalue AS varchar(max)), 1,
      PATINDEX('%[^0-9]%', CAST(ci.newvalue AS varchar(max)) + 'x') - 1) AS
INTEGER) AS sprint_id,
    MIN(cg.created) AS added_to_sprint_at
  FROM changegroup cg
  JOIN changeitem ci ON cg.id = ci.groupid
  WHERE ci.field = 'Sprint'
   AND ci.oldvalue IS NULL
   AND ci.newvalue IS NOT NULL
  GROUP BY cg.issueid, SUBSTRING(CAST(ci.newvalue AS varchar(max)), 1,
      PATINDEX('%[^0-9]%', CAST(ci.newvalue AS varchar(max)) + 'x') - 1)
),
sprint_scope_change AS (
  SELECT
    i.id AS issue_id,
    s.id AS sprint_id,
    s.name AS sprint_name,
    COALESCE(cfv.numbervalue, 0) AS story_points,
    i.created,
```

```
      ism.added_to_sprint_at,
      TRY_CONVERT(datetime, s.start_date) AS startdate,
      TRY_CONVERT(datetime, s.end_date) AS enddate,
      CASE
        WHEN ism.added_to_sprint_at IS NOT NULL AND s.start_date IS NOT
NULL AND
            TRY_CONVERT(datetime, ism.added_to_sprint_at) >
TRY_CONVERT(datetime, s.start_date)
            THEN 'Added'
        WHEN i.resolutiondate IS NOT NULL AND s.start_date IS NOT NULL AND
            TRY_CONVERT(datetime, i.resolutiondate) < TRY_CONVERT(datetime,
s.start_date)
            THEN 'Removed'
        ELSE 'Unchanged'
      END AS scope_change_status
    FROM jiraissue i
    JOIN issue_sprint_mapping ism ON i.id = ism.issueid
    JOIN ao_60db71_sprint s ON ism.sprint_id = s.id
    LEFT JOIN customfieldvalue cfv ON i.id = cfv.issue
      AND cfv.customfield = (SELECT id FROM customfield WHERE cfname =
'Story Points')
)
SELECT
    sprint_name AS "Sprint",
    scope_change_status AS "Sprint Scope Change",
    COUNT(*) AS "Issues",
    SUM(story_points) AS "Story Points"
FROM sprint_scope_change
WHERE issue_id IS NOT NULL
GROUP BY sprint_name, scope_change_status
ORDER BY sprint_name,
       CASE scope_change_status
         WHEN 'Unchanged' THEN 1
         WHEN 'Removed' THEN 2
         WHEN 'Added' THEN 3
       END;
```

Tables Used

- changegroup, changeitem: Used to detect when an issue was added to a sprint.
- jiraissue: Base issue data.
- ao_60db71_sprint: Sprint metadata (start/end dates).
- customfieldvalue: To extract "Story Points".

Key Logic

- Extracts Sprint ID from changeitem.newvalue using string functions.
- Compares the date the issue was added to the sprint vs the sprint start date to infer change type:
    - **Added**: Added after sprint start.
    - **Removed**: Resolved before sprint start.
    - **Unchanged**: Neither added late nor removed early.

Output Columns

- Sprint: Name of the sprint.
- Sprint Scope Change: Status of the issue (Added, Removed, Unchanged).
- Issues: Count of issues in each status.
- Story Points: Sum of story points for the respective category.
-

Customizations

- Replace 'Story Points' with your specific custom field name if different.
- Can add a WHERE clause to focus on specific sprints or timeframes.

5. Conclusion & Recommendations

Conclusions

The analysis successfully consolidated Jira sprint data using both API and backend SQL methods, enabling the extraction of key performance insights at board, sprint, and issue levels. Key findings include:

- Story Points Delivery Gap: Across sprints, a noticeable delta exists between committed and completed story points. This indicates potential issues in estimation, sprint scope changes, or delivery execution.

- Rolling Sprints Identified: The Sprint Rolling Status analysis highlighted a significant number of issues being carried over from previous sprints. This affects both team velocity and sprint predictability.
- Backlog Volatility: Frequent sprint scope changes (issues added or removed mid-sprint) suggest challenges in backlog grooming and planning consistency.
- Custom Fields & Schema Alignment: Custom field usage (e.g., story points via customfield_10106) must be standardized across boards and consistently maintained to support reliable metrics.
- Data Quality & Duplication: While merging data from multiple sources, some inconsistencies and duplicates were observed. These were cleaned in preprocessing but highlight the importance of centralized data governance.

Recommendations

To improve sprint delivery performance and analytics quality, the following actions are recommended:

1. Build Dashboard Visualizations
    - Implement visual dashboards showing sprint trends: velocity, committed vs completed story points, scope changes, and sprint carryovers.
    - Schedule automated exports of sprint data for stakeholders.
2. Standardize Jira Configuration
    - Align custom field IDs and board configurations across projects.
    - consolidate all data in one place that adheres to the JIRA 9.0 data model and standardize the process of querying the data from there.
3. Current Granularity v/s Expected Granularity of data
    - The above Data analysis was conducted on the following Granularity of data which is provided by JIRA 9.0 data model :

Project → Board → Sprint → Issues

- For further development, we need to incorporate Business Unit to Product and Product to Project correlation mapping. This will enhance the granularity of data, linking various **Business Units at the highest level of detail to their respective Sprints/Issues at the lowest level of detail** as shown below :

BU → Product **→Project →Board→Sprint→Issues**

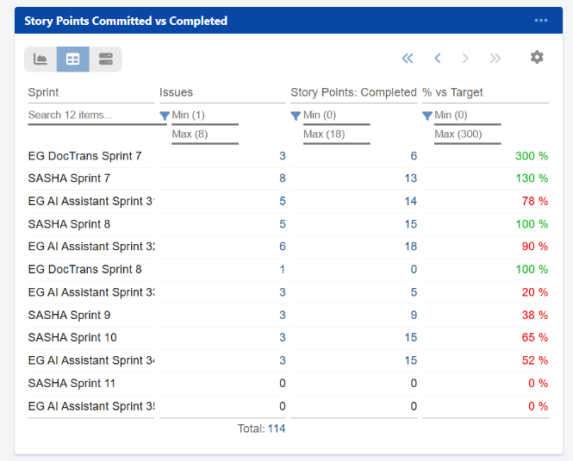
[New Level of Data granularity] [Existing level of Data granularity]

6.Results:

Using reverse engineering, we analysed the existing charts in the 'EG AI COE' scrum dashboard and designed SQL queries by examining the Pivot view of the data as shown below:

'SQL Query 1 – Story Points Committed vs Completed' referred Graph :

Pivot view of the data ('Story Points Committed vs Completed')

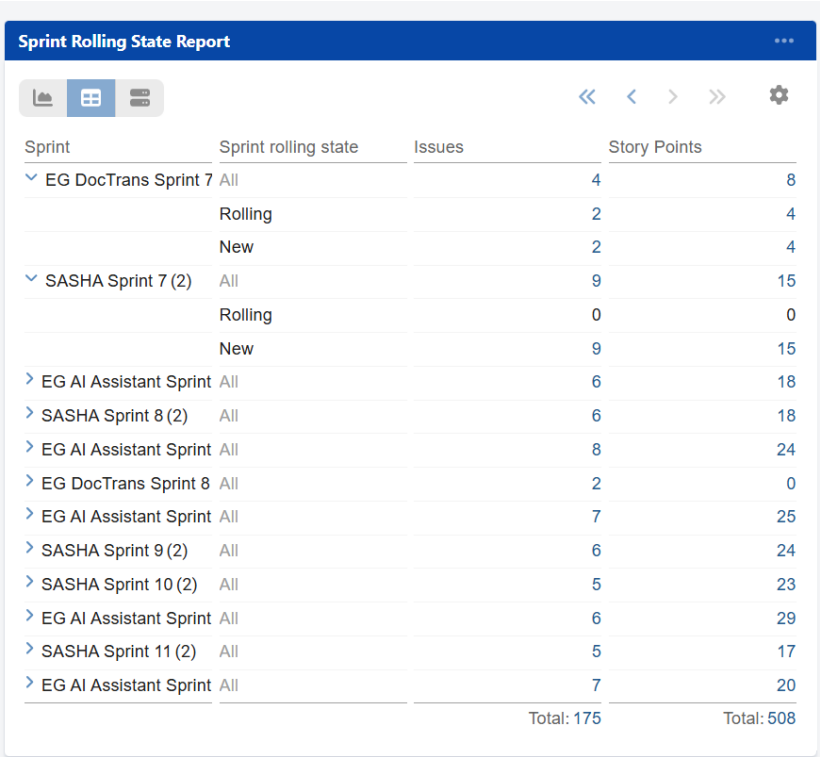


Story Points Committed vs Completed

Sprint	Issues	Story Points: Completed	% vs Target
Search 12 items...	Min (1) Max (8)	Min (0) Max (18)	Min (0) Max (300)
EG DocTrans Sprint 7	3	6	300 %
SASHA Sprint 7	8	13	130 %
EG AI Assistant Sprint 3	5	14	78 %
SASHA Sprint 8	5	15	100 %
EG AI Assistant Sprint 3	6	18	90 %
EG DocTrans Sprint 8	1	0	100 %
EG AI Assistant Sprint 3	3	5	20 %
SASHA Sprint 9	3	9	38 %
SASHA Sprint 10	3	15	65 %
EG AI Assistant Sprint 3	3	15	52 %
SASHA Sprint 11	0	0	0 %
EG AI Assistant Sprint 3	0	0	0 %
	Total: 114		

'SQL Query 2 – Sprint Rolling Status' referred Graph :

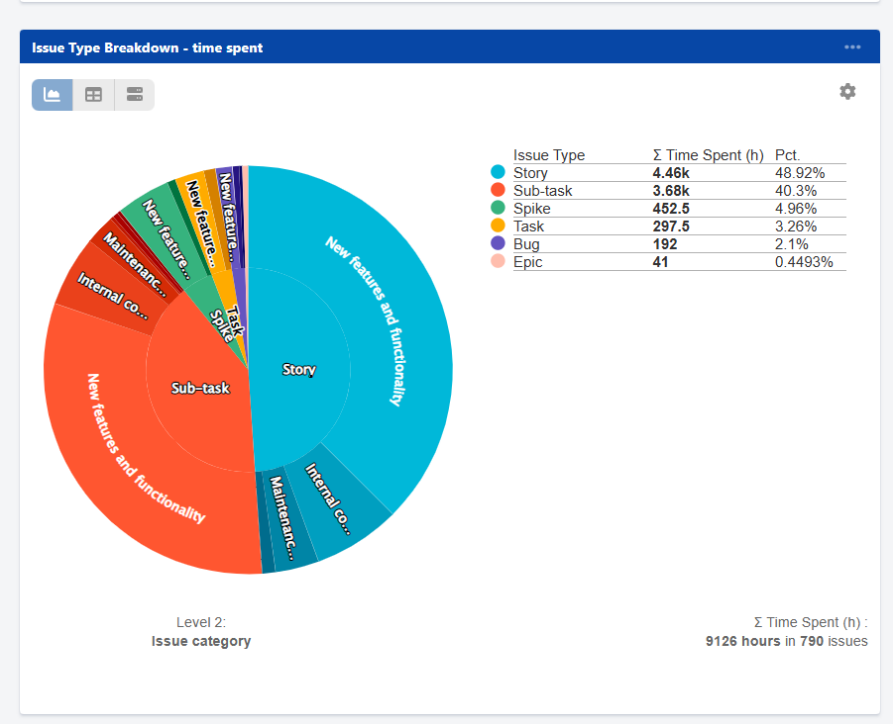
Pivot view of the data ('SQL Query 2 – Sprint Rolling Status')



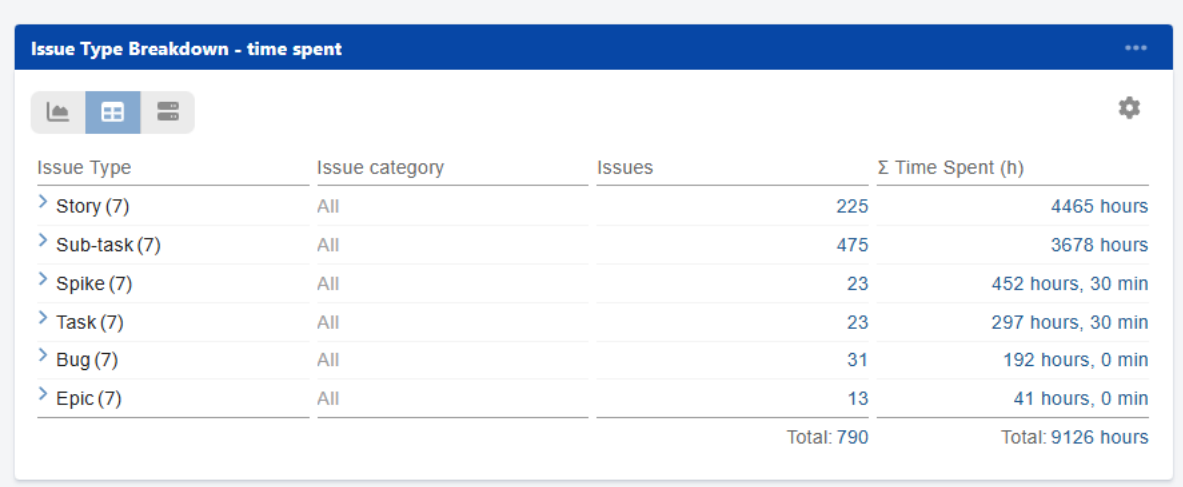
Sprint Rolling State Report

Sprint	Sprint rolling state	Issues	Story Points
EG DocTrans Sprint 7	All	4	8
	Rolling	2	4
	New	2	4
SASHA Sprint 7 (2)	All	9	15
	Rolling	0	0
	New	9	15
EG AI Assistant Sprint	All	6	18
SASHA Sprint 8 (2)	All	6	18
EG AI Assistant Sprint	All	8	24
EG DocTrans Sprint 8	All	2	0
EG AI Assistant Sprint	All	7	25
SASHA Sprint 9 (2)	All	6	24
SASHA Sprint 10 (2)	All	5	23
EG AI Assistant Sprint	All	6	29
SASHA Sprint 11 (2)	All	5	17
EG AI Assistant Sprint	All	7	20
		Total: 175	Total: 508

'SQL Query 3 – Issue Type Breakdown – Time Spent ' referred Graph :



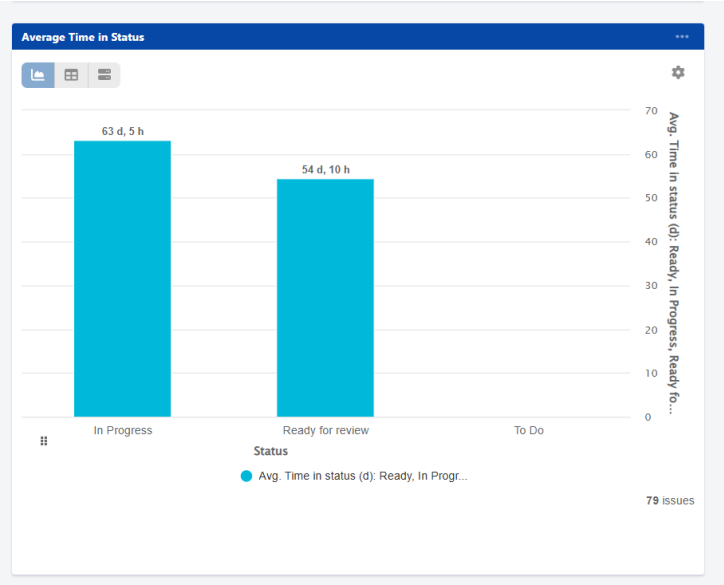
Pivot view of the data ('SQL Query 3 – Issue Type Breakdown – Time Spent')



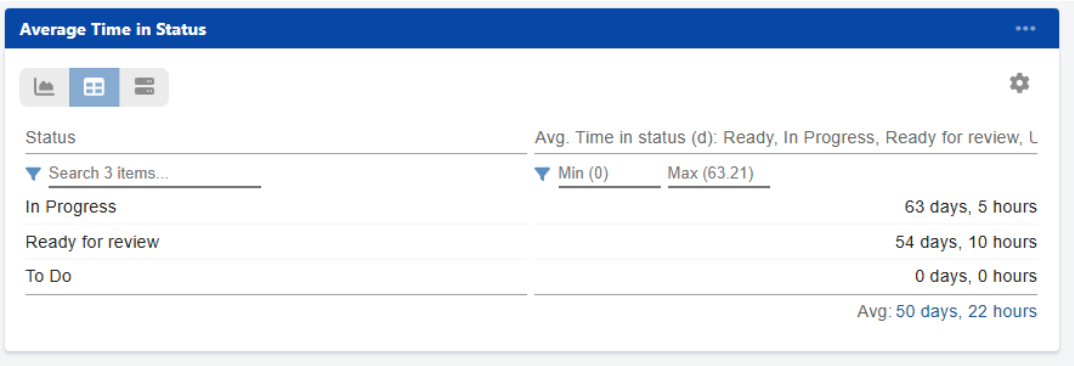
Issue Type Breakdown - time spent

Issue Type	Issue category	Issues	Σ Time Spent (h)
Story (7)	All	225	4465 hours
Sub-task (7)	All	475	3678 hours
Spike (7)	All	23	452 hours, 30 min
Task (7)	All	23	297 hours, 30 min
Bug (7)	All	31	192 hours, 0 min
Epic (7)	All	13	41 hours, 0 min
		Total: 790	Total: 9126 hours

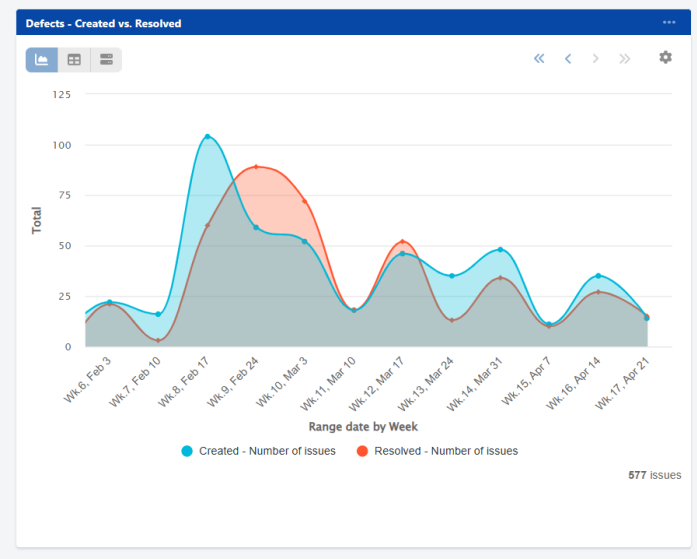
'SQL Query 4 – Average Time in Status' referred Graph :



Pivot view of the data ('SQL Query 4 – Average Time in Status')



'SQL Query 5 – Defects - Created vs Resolved' referred Graph:



Pivot view of the data ('SQL Query 5 – Defects - Created vs Resolved')

Defects - Created vs. Resolved

Range date by Week	Created - Number of issues	Resolved - Number of issues
Search 12 items...	Min (11) Max (104)	Min (3) Max (89)
Wk.6, Feb 3	22	21
Wk.7, Feb 10	16	3
Wk.8, Feb 17	104	60
Wk.9, Feb 24	59	89
Wk.10, Mar 3	52	72
Wk.11, Mar 10	18	18
Wk.12, Mar 17	46	52
Wk.13, Mar 24	35	13
Wk.14, Mar 31	48	34
Wk.15, Apr 7	11	10
Wk.16, Apr 14	35	27
Wk.17, Apr 21	14	15
	Total: 483	Total: 427

'SQL Query 6 – Sprint Scope Change' referred Graph:

Pivot view of the data ('SQL Query 6 – Sprint Scope Change')

Sprint Scope Change

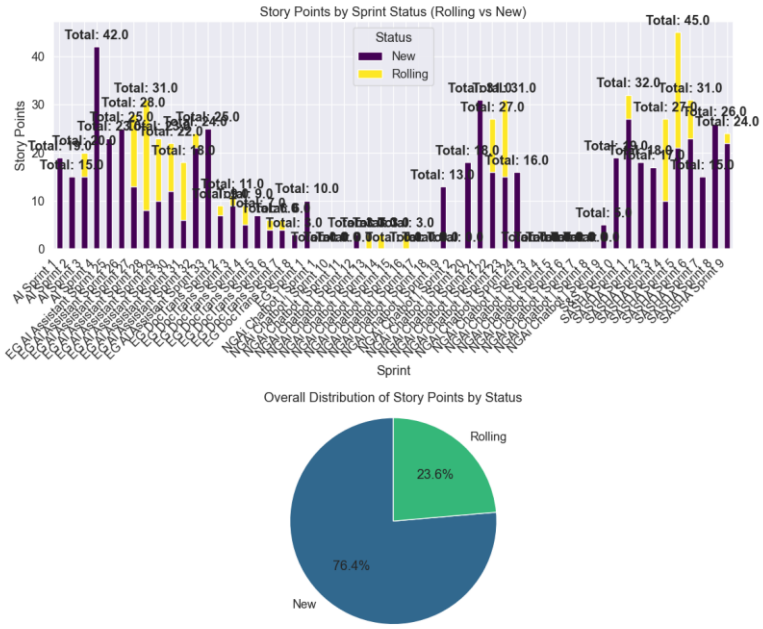
Sprint	Sprint scope change	Issues	Story Points
EG DocTrans Sprint 7 (3)	All	4	8
SASHA Sprint 7 (3)	All	12	15
EG AI Assistant Sprint 31 (3)	All	8	18
SASHA Sprint 8 (3)	All	16	18
EG AI Assistant Sprint 32 (3)	All	15	24
EG DocTrans Sprint 8 (3)	All	2	0
EG AI Assistant Sprint 33 (3)	All	9	25
SASHA Sprint 9 (3)	All	13	24
SASHA Sprint 10 (3)	All	22	23
EG AI Assistant Sprint 34 (3)	All	6	29
SASHA Sprint 11 (3)	All	5	17
EG AI Assistant Sprint 35 (3)	All	7	20
		Total: 547	Total: 570

Preprod JIRA API Data query results :

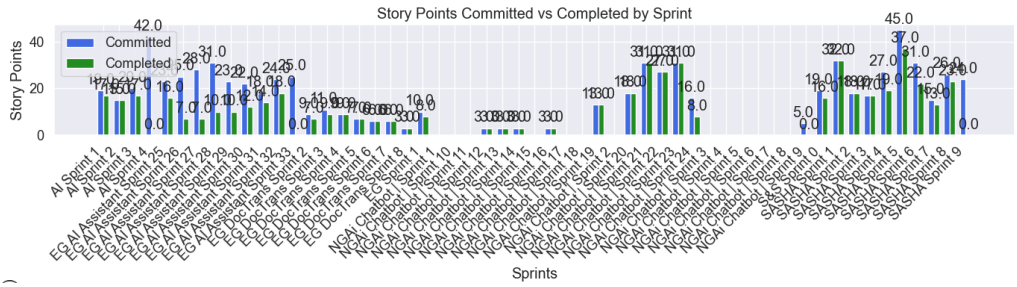
The Preprod data that was provisioned to us, was last refreshed at 01/04/2025 & it is synonymous to the Jira Prod, with which we initially ran our designed SQL queries.

The observations were as follows:

- Using ***'SQL Query 2 – Sprint Rolling Status',*** we observed an exact match in data metric values between the Preprod and Prod JIRA API data.



- Using 'SQL Query 1 – Story Points Committed', there is a slight difference in the values of data metrics for 'Story points committed' and 'Story points completed'. This is because we are comparing the current data in PROD with the data that was last refreshed on 01/04/2025 in preprod.



Inference:

'SQL Query 1 – Story Points Committed vs Completed' and ***'SQL Query 2 – Sprint Rolling Status'*** data was pulled from JIRA API due to unavailability of 'Board' level Data in 'Jira_Test' SSMS database instance

A Personal Access Token was generated & a Python integration was developed (JiraBoardSprint.py)

For the remaining four queries—***'SQL Query 3: Issue Type Breakdown – Time Spent'***, ***'SQL Query 4: Average Time in Status'***, ***'SQL Query 5: Defects – Created vs Resolved'***, and ***'SQL Query 6: Sprint Scope Change'***—we used the 'Jira_Test' DB instance and built queries based on the JIRA 9.0 data model.

Going forward, it is recommended to consolidate all data in one place that adheres to the JIRA 9.0 data model and standardize the process of querying the data from there.

7. Appendix

This section highlights the critical components of the implementation, both from the database query and the programmatic API-driven workflow.

SQL-Based Data Extraction

The following MS SQL queries were designed to extract and shape Jira sprint and issue data from the backend database (Jira 9.0 schema):

- **Sprint Master Extraction**:
  - Extracts sprint-related metadata including name, status, start/end dates, and goal.
  - Joins with relevant Agile tables to align boards and project metadata.
- **Issue Tracking Data**:
  - Aggregates issues assigned to each sprint, joined with resolution status and issue types.
  - Includes story point estimates where available via custom field lookups.
- **Story Points Analysis**:
  - Calculates story points committed vs completed per sprint.
  - Excludes sub-tasks to focus on deliverable stories.
- **Rolling Issue Identification**:
  - Uses SQL window functions (e.g., LAG()) to detect issues spanning multiple sprints.
  - Categorizes issues as either "New" or "Rolling".

These queries are optimized for historical analysis and periodic reporting.

Python + Jira API Integration

This section describes the modular Python code that interfaces directly with the Jira Cloud API via the Atlassian-python-API library.

- **Jira Initialization**:
  - Class-based wrapper (JiraManager) to safely initialize the Jira client using token-based authentication.
- **Paginated Board & Sprint Fetching**:
  - Functions to pull all boards and their associated sprints using paginated API calls.
  - Filters results to retain only Scrum boards and active sprint data.
- **Issue Extraction from Sprints**:
  - API endpoint used: /rest/agile/1.0/board/{boardId}/sprint/{sprintId}/issue
  - Captures issue metadata, including type, status, story points, and links to the board/sprint.
- **Data Merging & Master Table Creation**:
  - Combines board, sprint, and issue data into a unified DataFrame (master_table) for downstream analysis.
  - Handles duplication removal, column normalization, and derived field generation.
- **SQL-on-Pandas Analysis**:
  - Uses pandasql to apply SQL queries on the in-memory master_table DataFrame.
  - Key queries:
    - *Story Points Committed vs Completed*
    - *Sprint Rolling Status Report*
- **Output and Export**:
  - DataFrames are written to CSV files in the designated working directory for reproducibility and reporting.

File Outputs and Artifacts

The following files are generated as intermediate and final outputs:

File Path	Description
jira_board_data.csv	List of Scrum boards in the AI project
jira_sprint_data.csv	All sprints per board, filtered and cleaned
jira_master_dashboard_data.csv *(optional)*	Combined board-sprint-issue dataset for dashboarding
result1 (Story Points Analysis)	Sprint-level performance snapshot
result2 (Rolling Status Analysis)	Insight into sprint carryover patterns